Session-Aware Governance for Multi-Step LLM Service Workflows

Zhuozhen Gao[a]

[a] *Shenzhen University, Shenzhen, China*

Abstract

Multi-step LLM agents increasingly execute service workflows through tools and external services. Existing gateway and overload-control mechanisms usually govern individual requests, while the useful unit of work is a dependent session whose intermediate steps have no value if the session later fails. This mismatch creates cascade waste, unsafe replay risks, and ambiguous recovery behavior in overloaded agent services. We present PlanGate, a session-aware governance gateway for multi-step LLM service workflows. PlanGate combines hard admission for invalid or unsafe workflow declarations, adaptive step-0 admission, session commitment for declared plans, continuation-aware ReAct governance, client-cooperative recovery, idempotent side-effect protection, and Redis-backed shared state. In the core business-workflow experiment, adaptive governance reduces cascade failures by 31.8 sessions and wasted service time by 544,149 ms relative to disabling capacity admission under high-load failure conditions. Against a per-request admission baseline, PlanGate keeps completion comparable while reducing cascade waste; against an idempotency-only retry baseline, checkpoint recovery preserves progress beyond duplicate-write prevention. End-to-end HTTP+SQLite sanity checks complete 18 runs with zero duplicate side effects and consistent database state. On a six-node CloudLab deployment, Redis-backed shared state recovers 15/15 forced cross-gateway ReAct checkpoints with zero state misses, while memory-local state misses 15/15 cross-gateway resumes. The results show that session-aware governance reduces workflow waste and improves recovery correctness under constrained or shared backends, while elastic cloud providers define an important boundary for admission-control policies.

Keywords: LLM agents, service workflows, gateway governance, admission control, recovery protocol, MCP, distributed systems

1. Introduction

Large language model (LLM) agents are moving from single-turn text generation toward multi-step service workflows. A user request may trigger a sequence of tool calls, database updates, external API invocations, and model-backed reasoning steps. These workflows are increasingly exposed through protocols such as the Model Context Protocol (MCP), where agents call tools through gateway-like infrastructure [1]. In such settings, the gateway is no longer a passive transport component. It becomes a software control point responsible for admission, overload handling, state continuity, failure recovery, and side-effect safety.

A central difficulty is that conventional service governance operates at the wrong granularity. Rate limits, admission controllers, and overload policies usually decide whether to admit an individual request. Multi-step agent workflows, however, are valuable only at the session level. If an agent completes four tool calls and the fifth is rejected, the earlier calls may become useless. Worse, some earlier calls may already have committed external side effects, such as reservations, payments, ticket updates, or database writes. This creates a mismatch between the *governance unit*, which is often a request, and the *workload unit*, which is a dependent multi-step session.

This mismatch creates three practical failure modes. First, overloaded systems can admit sessions that later fail mid-execution, wasting tool calls, backend latency, model tokens, and service time. We refer to this as cascade waste. Second, recovery can be unsafe if a resumed session replays already completed side-effecting steps. Third, distributed gateway deployments can lose checkpoint state when a session fails on one gateway and resumes through another. These problems are software-system problems rather than model-quality problems: they arise from session state, resource governance, deployment topology, and workflow semantics.

This paper presents PlanGate, a session-aware governance gateway for multi-step LLM service workflows. PlanGate is implemented as an MCP-compatible reverse proxy between agent clients and tool backends. It separates three governance cases that are often conflated. First, malformed requests, invalid plans, security violations, and unsafe workflow declarations remain hard step-0 rejections. Second, capacity-driven step-0 admission is adaptive: it becomes stricter when backend congestion signals indicate that admitting more sessions would create downstream cascades, and looser when the backend has headroom. Third, continuation requests are governed with session progress in mind, because rejecting a late step wastes more prior work than rejecting a new session.

PlanGate also distinguishes the two dominant agent styles. Plan-and-Solve workflows can declare a plan or dependency structure before execution, allowing the gateway to apply budget, dependency, and safety checks at admission time. ReAct workflows reveal future tool calls incrementally, consis-

Email address: 2510268002@mails.szu.edu.cn (Zhuozhen Gao)

tent with the interleaved reasoning/action pattern introduced by ReAct [37], so the gateway cannot reserve an exact full plan upfront. For these sessions, PLANGATE uses continuation-aware governance and client-cooperative recovery: after a recoverable failure, the gateway restores checkpoint state and returns a resume payload, while the client continues the remaining workflow without replaying completed side-effecting steps. In distributed deployments, Redis-backed shared state allows checkpoint and reservation state to survive cross-gateway routing.

The paper is organized around five research questions. RQ1 asks whether session-aware governance reduces cascade waste in multi-step service workflows. RQ2 asks whether adaptive step-0 admission avoids the over-rejection of strict admission while still preventing overload-induced cascades. RQ3 asks whether ReAct recovery can resume from checkpoints without replaying completed side effects. RQ4 asks whether the system is deployable with bounded operator overhead and stable configuration behavior. RQ5 asks whether the design remains valid under real service backends and distributed gateway deployment.

Our evaluation is layered rather than based on a single benchmark. Controlled service workflows provide the main cascade and waste result. Mock and vLLM runs isolate backend congestion. Provider smoke tests characterize elasticity. HTTP+SQLite workflows check persisted side effects and idempotency. CloudLab checks distributed shared state. Overhead, ablation, sensitivity, and protocol studies address deployability. The evaluation keeps performance, correctness, deployability, and scope results separate.

Three findings summarize the empirical results. First, under constrained business-workflow load, adaptive governance reduces cascade failures by 31.8 sessions and wasted service time by 544,149 ms relative to disabling capacity admission. Second, ReAct recovery and idempotency checks preserve side-effect safety: the recovery protocol passes 11/11 edge cases, and HTTP+SQLite sanity runs complete with zero duplicate side effects and consistent database state. Third, distributed correctness depends on shared state: in CloudLab, Redis-backed state recovers 15/15 forced cross-gateway ReAct checkpoints with zero state misses, while memory-local state misses 15/15 cross-gateway resumes. At the same time, cloud-provider experiments show that elastic backends can invert admission trade-offs, so PLANGATE should be interpreted as a constrained/shared-backend governance framework rather than a policy that dominates in every environment.

1.1. Research Contributions

This paper makes the following contributions:

- **Problem formulation.** We formulate session-aware governance as a software-systems problem for multi-step LLM service workflows, identifying cascade waste, side-effect replay, and distributed checkpoint loss as core failure modes.
- **System design.** We design PLANGATE, an MCP-compatible governance gateway that combines hard safety admission, adaptive capacity admission, session commitment for declared plans, continuation-aware ReAct governance, client-cooperative recovery, idempotency, and Redis-backed shared state.
- **Implementation and reproducibility.** We implement PLANGATE with MCP, HTTP service, SQLite, Redis, vLLM, and cloud-provider adapters, and provide statistical summaries, effect-size tables, scoped findings, and lightweight replay checks.
- **Empirical results.** We evaluate PLANGATE across controlled workflows, request-level and idempotency-only baselines, mock and vLLM backends, cloud-provider scope cases, real HTTP+SQLite services, and CloudLab distributed deployment.

1.2. Paper Organization

The rest of the paper develops this argument as a software-systems study. It first introduces the background and problem statement, then presents the PlanGate architecture and design mechanisms, followed by implementation details, evaluation methodology, experimental results, discussion, threats to validity, related work, and conclusion.

2. Background and Motivation

2.1. Multi-Step LLM Service Workflows

LLM agents increasingly act as workflow coordinators. Instead of producing a single response, an agent may search for information, call a pricing service, reserve inventory, update a ticket, authorize a payment, and summarize the result. Each step can be implemented as an MCP tool call, an HTTP API call, a database update, or a model-backed computation. From a software-systems perspective, this workload resembles a service workflow: a user-facing task is decomposed into an ordered set of service invocations whose correctness depends on control flow, data dependencies, and side-effect management, echoing long-standing service-oriented computing, workflow-pattern, and saga concerns [22, 23, 30, 14].

These workflows differ from traditional independent RPC workloads in two ways. First, tool invocations within a session are not interchangeable requests. A later call depends on earlier observations, and a rejected later step may invalidate all earlier work. Second, some tools are side-effecting. A workflow may reserve a flight, create an order row, apply a credit, or update a support ticket. Retrying or resuming such workflows requires idempotency and checkpoint semantics, not only request retransmission. These properties make multi-step agent workloads a natural target for service governance rather than a pure model-evaluation problem.

2.2. Agent Styles and Available Information

PLANGATE targets two common agent execution styles. We use Plan-and-Solve to denote a declared-plan workflow mode, not a claim about a specific prompting benchmark. Such workflows expose a plan before execution. The plan may include a step list, dependency graph, budget declaration, or other metadata that allows a gateway to reason about resource needs before the first tool executes. This enables stronger admission semantics: the gateway can reject malformed or unsafe declarations, check budget feasibility, and bind the admitted session to a commitment scope.

ReAct-style workflows expose less information upfront. The agent decides the next tool call after observing the previous result, so the gateway cannot know the full future plan at step 0 [37]. Treating ReAct as if it had a declared plan would overstate what the infrastructure can guarantee. Instead, ReAct governance must be incremental: the gateway observes session progress, assigns continuation value to already completed work, and provides a recovery point when a recoverable failure occurs. In PlanGate, ReAct recovery is client-cooperative: the gateway restores checkpoint state and returns a resume payload, while the client remains responsible for issuing the remaining tool calls.

2.3. Why Request-Level Governance Is Insufficient

Most existing gateway policies make decisions per request. This is appropriate for stateless RPC traffic, but it is incomplete for dependent sessions. A request-level limiter can reject a tool call at step 5 even if steps 1–4 have already consumed backend capacity. A delay-sensitive price controller can increase prices mid-session because the system became congested after the session started. A local checkpoint store can recover a session only if the resume request reaches the same gateway process. These decisions may be locally reasonable while globally wasteful at the session level.

The mismatch produces a characteristic failure pattern: a session is admitted, consumes partial work, and then fails before completion. The failure is worse than a step-0 rejection because the system has already consumed service time, tokens, and possibly durable side effects. The goal of session-aware governance is not to eliminate rejection. Rejection is necessary under overload and for invalid workflows. The goal is to move unavoidable capacity rejection earlier, preserve admitted progress when possible, and prevent unsafe replay during recovery.

2.4. Motivating Service-Engineering Requirements

The service-engineering requirements for such workloads are broader than throughput and latency. A useful gateway must preserve workflow semantics, expose a clear recovery contract, keep side effects idempotent, and remain deployable in multi-gateway systems. This motivates four requirements:

- **R1: Session-level admission and continuation.** Admission should distinguish new sessions from continuation steps and should account for the increasing waste of late-session rejection.
- **R2: Explicit recovery contract.** Recovery should restore progress without replaying completed side-effecting steps or automatically executing future tools without client intent.
- **R3: Idempotent side-effect integrity.** Service workflows must use idempotency and final-state checks so that retries and resumes do not duplicate durable operations.
- **R4: Distributed checkpoint availability.** In multi-gateway deployments, reservations and checkpoints must be available across gateway boundaries, or cross-node resume will lose state.

These requirements motivate an evaluation hierarchy that separates controlled waste-reduction experiments from side-effect, backend-realism, and distributed-state evidence.

3. Problem Statement and Research Questions

3.1. System Model

We consider a service workflow initiated by an LLM agent client. A workflow session consists of a sequence of tool invocations $s = (r_1, r_2, \ldots, r_k)$ routed through a gateway to one or more backend services. Each request may consume service time, model tokens, queue capacity, and possibly durable side effects. A session succeeds only if the workflow reaches a valid terminal state. A session can be rejected at step 0 before any tool execution, or it can fail after admission because a later step is rejected, times out, or encounters a recoverable interruption.

We distinguish three classes of rejection. *Hard rejection* applies to malformed requests, invalid workflow declarations, security violations, unsafe plan amendments, or inconsistent recovery tokens. These failures should remain strict because admitting them would violate system invariants. *Capacity rejection* applies when backend load suggests that admitting more work would cause queue buildup or downstream cascade. This rejection should be adaptive because the appropriate policy depends on actual congestion. *Continuation rejection* applies to already admitted sessions; it is expensive because it can waste prior work and should therefore be governed differently from new-session admission.

3.2. Failure Metrics

The primary failure mode is cascade waste. A cascade failure occurs when an admitted session fails after completing at least one step. The wasted work includes completed tool calls, service time, model tokens, and side-effect management overhead that no longer contribute to a successful workflow. We also track step-0 rejection, because moving rejection earlier can reduce waste but may reduce availability. These metrics create a trade-off: a policy that admits everything can maximize raw success under elastic backends but may create large cascade waste under hard capacity; a policy that rejects too aggressively can reduce waste while denying workflows that could have completed.

For recovery, we track whether the gateway returns a valid resume payload, whether the client can continue after resume, whether completed steps are replayed, and whether durable side effects remain unique. For distributed deployments, we track cross-gateway resume, state misses, duplicate admissions, and

Table 1: Core service-governance metrics used in the paper.

Metric	Meaning
Cascade failure	An admitted session fails after at least one completed step.
Wasted service time	Service time consumed by sessions that do not reach a valid terminal state.
Step-0 rejection	Rejection before workflow work begins.
Duplicate side effect	Repeated durable operation under the same logical workflow.
State miss	A resume path cannot find checkpoint or reservation state.

final database consistency. These metrics reflect service correctness rather than model accuracy. Table 1 summarizes the core terms used throughout the evaluation.

3.3. Formal Metric Definitions

The metrics used in the evaluation are measurement definitions, not an optimization objective. Let a workflow session be an ordered sequence of tool requests

$$s = \langle r_{s,0}, r_{s,1}, \ldots, r_{s,T_s-1} \rangle. \tag{1}$$

Let p_s be the number of completed tool steps before the terminal event of session s, let z_s be the terminal outcome, and let $t_{s,i}$ be the service time spent by step i. We define cascade failure as

$$\mathrm{Cascade}(s) = \mathbb{1}[p_s > 0 \wedge z_s \in \{\mathrm{fail}, \mathrm{timeout}, \mathrm{rejected}\}]. \tag{2}$$

The wasted service time attributed to session s is

$$W(s) = \mathrm{Cascade}(s) \sum_{i=0}^{p_s-1} t_{s,i}, \qquad W(\mathcal{S}) = \sum_{s \in \mathcal{S}} W(s). \tag{3}$$

For durable side effects, let O be the set of logical side-effect identifiers and n_o be the number of committed rows or operations observed for identifier o. Duplicate side effects are counted as

$$D = \sum_{o \in O} \max(0, n_o - 1). \tag{4}$$

For recovery, if c_s is the completed checkpoint prefix restored for session s, the avoided replay count is

$$A(s) = \mathbb{1}[\mathrm{Recovered}(s)]\, c_s. \tag{5}$$

These definitions make the evaluation explicit: reducing cascade waste is different from maximizing raw success, and recovery correctness requires both progress restoration and duplicate-side-effect avoidance.

3.4. Research Questions

The paper is organized around five research questions:

RQ1. Does session-aware governance reduce cascade failures and wasted service work in multi-step service workflows under constrained backend capacity?

RQ2. How does adaptive capacity-driven step-0 admission trade off early rejection against mid-session cascade compared with always-strict and no-capacity-step0 policies?

RQ3. Can client-cooperative ReAct recovery resume from checkpoints, continue remaining workflow steps, and avoid replaying completed side-effecting operations?

RQ4. Does the gateway introduce bounded operator overhead, and do its qualitative signals persist under configuration and component variations?

RQ5. Does the design hold across external settings: HTTP+SQLite services, model-backed backends, cloud providers, and CloudLab?

The research questions separate effectiveness, correctness, deployability, and scope. For example, the HTTP+SQLite and CloudLab studies are not used to argue raw performance superiority; they check side-effect safety and distributed correctness. Conversely, the controlled business-workflow experiments provide the main basis for cascade-waste reduction.

3.5. Scope and Non-Goals

PlanGate is not a model-quality benchmark and does not attempt to improve the reasoning accuracy of an LLM agent. It governs service workflow execution after the agent chooses tools. The system also does not claim production Redis high availability, Byzantine security against malicious clients, or fully automatic server-side recovery. Recovery is client-cooperative by design: the gateway exposes a safe resume point, and the client continues the remaining workflow. Finally, the paper does not claim that one admission policy dominates in all environments. Cloud-provider experiments show that elastic providers can hide backend congestion and change the best raw-success trade-off.

4. PlanGate Overview

4.1. Architecture

PlanGate is a gateway layer between agent clients and backend tools. The client sends MCP-compatible tool calls with session metadata. The gateway classifies each request, applies admission and recovery logic, and forwards admitted calls to the backend. The backend may be a mock service, a vLLM-backed tool service [18], a cloud-provider-backed tool service, or an HTTP business service backed by SQLite. In distributed deployments, multiple gateway processes share reservation and checkpoint state through Redis. Figure 1 shows the architecture and workflow lifecycle.

The architecture has six logical components. The admission controller handles hard workflow rejection and adaptive capacity admission. The session manager tracks active sessions, commitments, and continuation state. The ReAct checkpoint manager records completed progress for recoverable sessions. The recovery protocol returns explicit resume payloads and requires client-side continuation. The idempotency layer protects side-effecting tools from duplicate writes. The shared-state adapter stores reservations and checkpoints either in process-local memory or in Redis.

4.2. Request Lifecycle

At the start of a workflow, the gateway first applies hard validation. Invalid plans, malformed metadata, unsafe amendments, and inconsistent recovery tokens are rejected before backend execution. If the request is a new session and the only concern is backend capacity, PlanGate applies adaptive step-0 admission. The policy is intentionally not always strict: strict admission can over-reject when the backend has headroom or when a cloud provider absorbs load, while disabling capacity admission can create large cascade waste under constrained backends.

For a declared workflow, PlanGate records a session commitment and tracks the admitted scope. For a ReAct workflow, the gateway records progress as the client issues tools. When a recoverable failure occurs, the checkpoint manager marks the session recoverable and records the last completed safe point. A resume request with the same session identity returns a payload indicating that the session was recovered, the current step is greater than zero, and client continuation is required. The gateway does not execute future tools by itself.

For side-effecting services, the tool layer attaches idempotency keys and checks database consistency. This is essential because recovery without idempotency can duplicate external operations. The evaluation later checks this path with HTTP business services backed by SQLite, but the overview only defines the control path.

4.3. Deployment Modes

PlanGate supports single-gateway and multi-gateway deployment. In a single-gateway setting, in-memory state is sufficient for local experiments and small deployments. In a multi-gateway setting, request routing may send a failure and its resume request to different gateway processes. Memory-local state cannot reliably recover such sessions because the checkpoint may exist only in the gateway that handled the failure. Redis-backed shared state addresses this by externalizing reservation and checkpoint lookup.

4.4. Design Boundary

Table 2 summarizes component responsibilities and limits. The evaluation later maps these components to concrete studies. The key point is that PlanGate's contribution is not "always reject earlier" or "always admit more." Its contribution is a session-aware control plane that makes admission, continuation, recovery, and shared-state decisions explicit and measurable. Under constrained or shared backends this can reduce cascade waste; under elastic cloud-provider backends the best raw-success policy can differ, so deployment context matters.

5. Design

5.1. Design Principles

PlanGate follows three design principles. First, workflow validity and capacity pressure are separated. Malformed metadata, invalid plans, unsafe amendments, and inconsistent recovery tokens remain hard failures that are rejected before backend execution. Capacity pressure is governed separately because its correct handling depends on recent load, queueing behavior, and provider elasticity.

Second, session progress changes admission cost. Rejecting a new session before work begins wastes little service work; rejecting a continuation step after several completed tools can waste service time, tokens, and side-effect management. PlanGate therefore treats new-session admission and continuation admission differently, without assuming that every continuation must be admitted.

Third, recovery is explicit and idempotent. The gateway exposes a resume point but does not replay completed side-effecting operations or execute future tools on behalf of the client. The client-cooperative contract keeps recovery observable, while idempotency keys protect durable operations from duplicate writes.

5.2. Admission and Commitment

For declared-plan workflows, PlanGate uses admission as a commitment point. A client may provide a session identifier, declared steps, dependencies, budget information, or amendment intent. The gateway validates this metadata before backend execution and rejects requests that violate the declared workflow contract. When the workflow is admitted, the session manager records an active commitment and checks later continuation steps against that scope. This mechanism applies to workflows that expose enough structure for upfront reasoning; it is not treated as a full-plan guarantee for ReAct sessions.

5.3. Adaptive Capacity Admission

The adaptive module only applies to capacity-driven decisions; hard validation is handled before this path. In the submitted implementation, the adaptive gate uses three active runtime signals:

$$session\_cap\_load = active\_sessions/session\_cap,$$
$$react\_step0\_load = react\_step0\_inflight/react\_step0\_limit.$$

Governance intensity is a separate input signal exposed to the gate; in the submitted artifacts it is produced online by a runtime intensity tracker over the gateway's price state rather than inferred from reported outcomes. The gate classifies a state as red when any active signal crosses its red threshold, green when all active signals remain below their green thresholds, and yellow otherwise. Hard rejection for malformed metadata, invalid plans, unsafe amendments, or inconsistent recovery tokens is evaluated before this adaptive path and is therefore unaffected by the adaptive capacity gate.

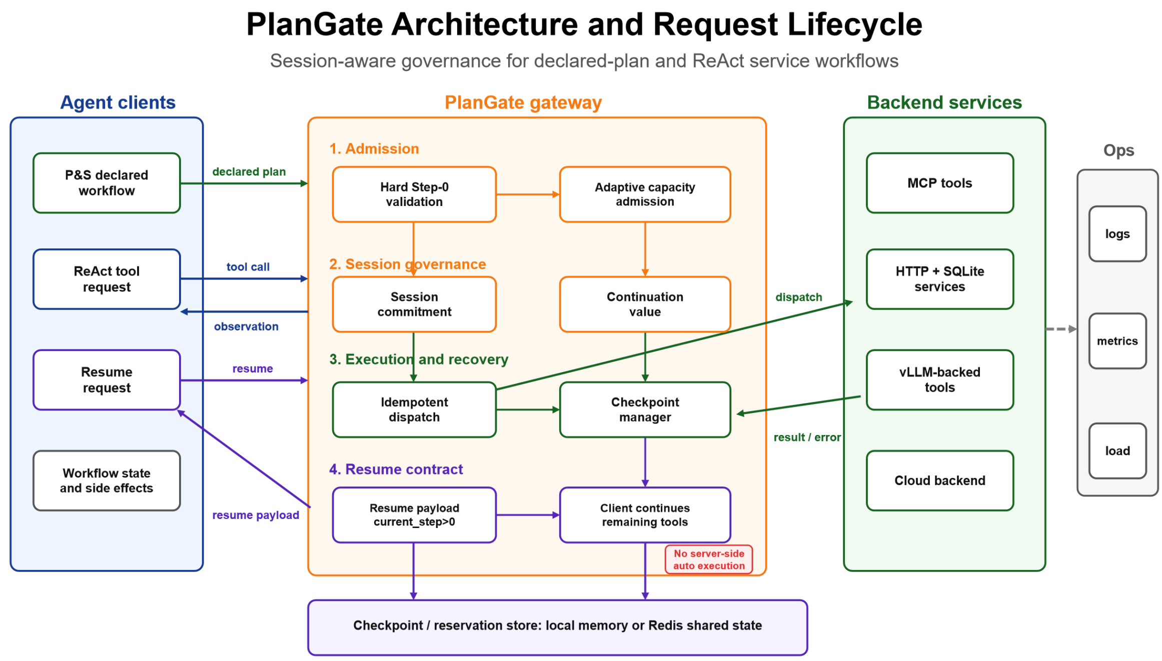

Figure 1: PlanGate architecture and workflow lifecycle. The gateway sits between agent clients and tool backends, separating hard validation, adaptive capacity admission, session tracking, recovery, idempotency, and shared-state lookup.

Table 2: PlanGate component responsibilities and design boundaries.

Component	Responsibility	Boundary
Adaptive admission	Separates hard rejection from capacity-driven admission and controls the early-reject versus cascade trade-off.	Does not guarantee raw-success dominance.
Session commitment	Uses declared workflow information for stronger admission and continuation semantics.	Applies within declared scope and valid metadata.
ReAct checkpointing	Preserves completed ReAct progress after recoverable interruption.	Does not auto-execute future tools.
Recovery protocol	Exposes a resume point and requires explicit client continuation.	Not transparent server-side replay.
Idempotency layer	Prevents duplicate durable side effects during retry and resume.	Tested workflow-side idempotency, not complete malicious-client security.
Shared-state adapter	Shares reservations and checkpoints across gateway processes.	Not production Redis HA or fault tolerance.

Table 3: Default adaptive-admission state thresholds.

Signal / parameter	Green / default	Red / closed
Governance intensity	< 0.20	≥ 0.65
Session-cap load	< 0.70	≥ 0.90
ReAct step-0 inflight load	< 0.70	≥ 0.90
Cascade-rate hook	–	≥ 0.15
Recoverable-error hook	–	≥ 0.20
Green wait	200 ms	–
Yellow wait	50 ms	–
Red wait	–	0 ms

Table 3 reports the default thresholds used by the adaptive admission module. The current experiments use governance intensity, session-cap load, and ReAct step-0 inflight load as active signals. Cascade-rate and recoverable-error thresholds are implemented as extension hooks and remain zero in the submitted runs.

Algorithm 1 summarizes the admission path. The key distinction is that hard rejection is evaluated before the adaptive gate, while continuation and recovery requests use session state rather than the same rule as new-session admission.

The design is evaluated through three admission policies. A *strict* policy disables adaptive classification and rejects new sessions immediately under capacity pressure. A *no-capacity-step0* policy disables capacity-driven step-0 rejection entirely. The *adaptive* policy keeps hard rejection but changes capacity admission through the green/yellow/red wait policy of 200/50/0 ms for new sessions. This comparison is necessary because constrained local backends and elastic cloud providers can prefer different raw-success trade-offs.

5.4. ReAct Checkpointing and Recovery

ReAct workflows choose tools incrementally, so PlanGate cannot reserve a complete future plan. Instead, each tool call carries session and step metadata. The gateway records completed progress after successful tool execution and uses this state to distinguish new work from continuation work.

Algorithm 1 Session-aware admission and continuation

Require: Request r, session state σ, backend pressure ρ, policy π
if ViolatesHardInvariant(r, σ) **then**
  return Reject(hard)
end if
if IsRecoveryRequest(r) **then**
  return ReactRecovery(r, σ)
end if
if IsContinuation(r) **then**
  if not HasSessionState($r.session, \sigma$) **then**
    return Reject(state-miss)
  end if
  $v \leftarrow$ ContinuationValue($r.session, \sigma$)
  if Overloaded(ρ) **and not** ContinuationAdmissible(v, ρ) **then**
    return Reject(continuation)
  end if
  return ForwardAndCheckpoint(r)
end if
if π = no_capacity_step0 **then**
  return AdmitNewSession(r)
else if π = strict **and** CapacityPressure(ρ) **then**
  return Reject(capacity-step0)
else if π = adaptive **and** AdaptiveGateClosed(ρ, σ) **then**
  return Reject(capacity-step0)
else
  return AdmitNewSession(r)
end if

Algorithm 2 Client-cooperative ReAct recovery

Require: Resume request r with session id sid, checkpoint store C
$c \leftarrow$ LookupCheckpoint(C, sid)
if $c = \varnothing$ **then**
  return Reject(state-miss)
end if
if $c.status$ = active **then**
  return Reject(live-checkpoint)
end if
if $c.status \neq$ recoverable **then**
  return Reject(non-recoverable)
end if
RestoreSession($sid, c.completedPrefix$)
$payload \leftarrow$ {recovered: true, mode: react-client-cooperative,
  current-step: $c.nextStep$, requires-client-continuation: true,
  avoided-replay-steps: $c.completedPrefix$}
return $payload$
▷ The gateway restores state but does not execute future tools.

When a recoverable failure occurs, PlanGate marks the session as checkpointed. A resume request with the same session identity and explicit recovery mode restores the checkpoint and returns a payload describing the recovered state, current step, and requirement for client continuation. The gateway does not execute future tools during recovery. This boundary prevents hidden server-side replay and keeps future tool choice under client control. Figure 2 summarizes this client-cooperative protocol.

Algorithm 2 gives the recovery contract. It deliberately returns a resume payload rather than running the next tool call inside the gateway.

5.5. *Side-Effect Integrity*

Service workflows often contain durable operations. The mechanism assumes that side-effecting tools expose stable idempotency keys and side-effect identifiers. Normal retry should return the same logical operation result rather than create a duplicate write; recovery should not replay completed side-effecting steps. The evaluation uses HTTP services backed by SQLite to check this property, but the design itself only requires idempotency keys, checkpointed progress, and final-state validation.

5.6. *Distributed Shared State*

In a single gateway, session state can be kept in memory. In a multi-gateway deployment, a failure and its resume request may reach different gateway processes, so process-local checkpoints are insufficient. PlanGate therefore supports a shared-state adapter for reservations, checkpoints, and recovery metadata. Redis is used as a shared lookup service in the current implementation, while memory-local state remains a diagnostic control. This design supports cross-gateway recovery but does not claim production Redis high availability, node-crash recovery, or fault-tolerant store management.

6. Implementation

PlanGate is implemented in Go as an MCP-compatible gateway. It exposes HTTP handlers for tool forwarding, session metadata, admission, and recovery. The command-line entry point wires together admission configuration, recovery settings, backend routing, and state-store selection. Mechanism code is separated from experiment scripts, so mock, vLLM, cloud-provider, HTTP-service, and CloudLab experiments exercise the same gateway path.

Requests carry session metadata through headers and JSON fields, including session identity, workflow mode, step index, optional declared-plan information, idempotency keys, and recovery mode. The gateway translates these fields into admission checks, session-manager operations, checkpoint updates, and backend forwarding decisions. The admission modules implement adaptive capacity control and governance intensity; the session manager tracks active sessions, completed progress, recovery state, and session-cap accounting. Unit tests cover resume semantics, live-checkpoint rejection, absence of future-tool execution, and session-cap leak prevention.

The state layer supports in-memory and Redis-backed stores. The Redis-backed store externalizes reservations and checkpoints for multi-gateway deployments, while the in-memory store is used for local runs and diagnostic controls. To test real service boundaries, the artifact includes HTTP business services backed by SQLite and an MCP JSON-RPC wrapper that calls those services rather than writing the database directly. The SQLite service uses WAL mode, foreign keys, idempotency records, and final-state checks. Backend adapters support controlled mock services, local OpenAI-compatible vLLM endpoints, and cloud-provider APIs.

Experiment scripts generate per-experiment result bundles and a consolidated statistical package. The pipeline stores check metadata, summary CSVs, aggregate CSVs, and a table that links numerical findings to their intended use. Detailed script names, CSV schemas, and deployment recipes are documented in the submitted artifact rather than repeated in the main text.

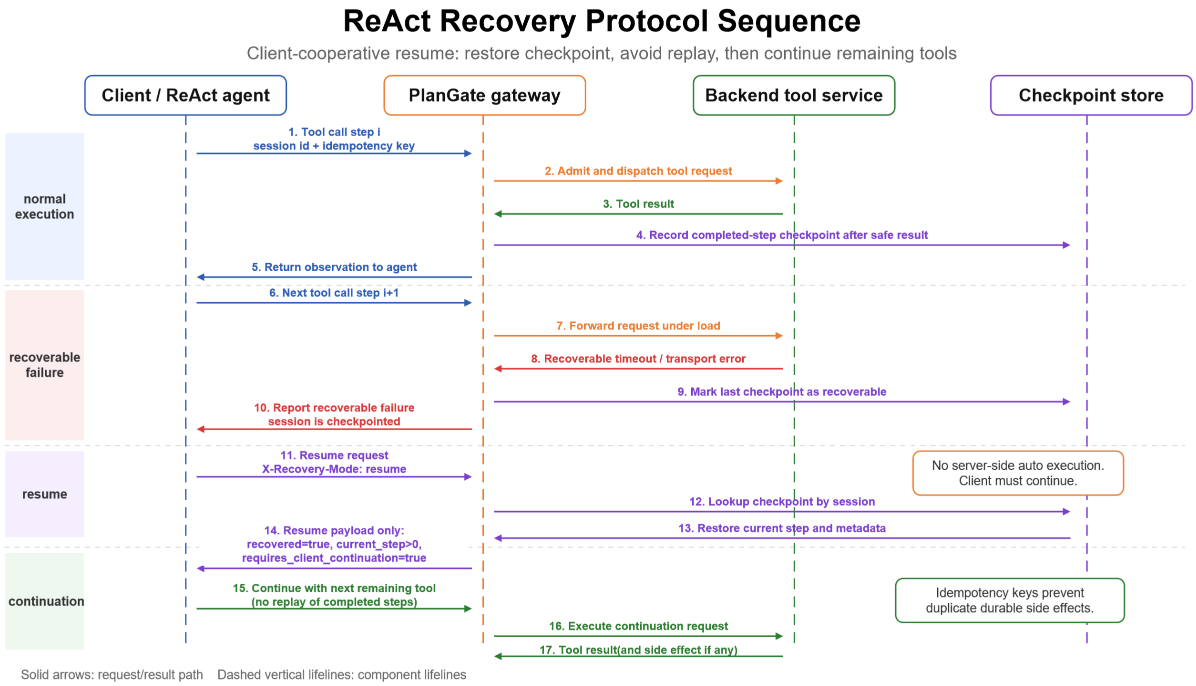

Figure 2: Client-cooperative ReAct recovery protocol. The gateway restores checkpointed progress and returns a resume payload; the client then continues the remaining workflow steps explicitly.

7. Evaluation Methodology

7.1. Evaluation Strategy

The evaluation is finding-driven. Each research question is matched to a study with an explicit role and scope. Controlled business-workflow experiments provide the main causal results for waste reduction. Recovery probes and HTTP+SQLite workflows check correctness and side-effect safety. vLLM experiments exercise model-backed backend behavior. Cloud-provider runs characterize elasticity limits. CloudLab experiments check distributed checkpoint availability. Overhead, ablation, sensitivity, and usability studies support deployability and design interpretation.

This structure separates performance findings from correctness checks. CloudLab probes check cross-gateway recovery and shared checkpoint availability, not throughput superiority. Cloud-provider runs define scope because provider elasticity can invert admission trade-offs observed under constrained local backends.

7.2. Policies, Workloads, and Metrics

The main policy comparisons use four variants: strict, adaptive, no-capacity-step0, and adaptive-react-recovery. Strict tests aggressive early rejection; no-capacity-step0 removes capacity-driven step-0 rejection; adaptive tests pressure-aware admission; adaptive-react-recovery adds client-cooperative ReAct recovery. The evaluation avoids treating these variants as a universal ranking because their behavior depends on backend elasticity.

The workload layers are scoped by purpose. Controlled business workflows are the core performance setting. Mock and vLLM layers isolate backend congestion and model-backed execution [18]. HTTP+SQLite workflows exercise persistent side effects and idempotency. Cloud-provider smoke tests check compatibility and admission-policy scope. MCP-Bench-derived smoke and BurstGPT-shaped replay provide workflow-shape and arrival-pattern realism [33, 32], but they are not used as headline performance rankings. Table 4 summarizes the main studies.

The primary performance metrics are completed sessions, cascade failures, step-0 capacity rejections, wasted service time, effective throughput, and tail latency. Recovery metrics include resume attempts, recovered sessions, avoided replay steps, continuation after resume, duplicate side effects, and replayed completed side effects. Distributed metrics include state misses, duplicate admissions, cross-gateway resumes, and final database consistency.

The request-level baseline is intentionally operational rather than a strawman. It forwards each tool call through a per-request proxy that admits if the current backend inflight count is below a fixed request limit and otherwise waits briefly before returning a request-level rejection. In the submitted artifact, the proxy uses a 30-request inflight limit and a 50 ms wait budget. The decision uses only per-request queue pressure: it ignores session identifier, step index, completed prefix, continuation value, and checkpoint state for admission. Idempotency keys are still forwarded so durable-write safety remains a fair comparison.

7.3. Claim Control and Reproducibility

The paper uses the statistical-summary table to control numerical statements. It records each JSS finding with source

Table 4: Experimental setup summary. Smoke and correctness studies are not interpreted as headline performance rankings.

Study	Artifact	Backend / deployment	Scale / repeats	Primary metrics	Role
Core workflow	`business_workflow_service_v3`	Controlled P&S/ReAct service workflow	C={20, 50, 100}, F={0, 0.1, 0.2}, 5 repeats	Cascade, wasted service time	Main result
Request-level baseline	`request_level_baseline_v1`	Same workflow harness, per-request queue admission only	C={50, 100}, F={0, 0.2}, 5 repeats	Cascade, wasted service time, request rejects	Baseline
Idempotency-only baseline	`idempotency_only_retry_baseline_v1`	HTTP+SQLite workflow, no resume	C={10, 20}, F=0.1, 3 repeats	Duplicate side effects, replayed steps	Recovery baseline
Full ablation	`adaptive_step0_react_recovery_full_v1`	Mock and local vLLM layers	84 rows	Trend checks, recovery, cascade	Mechanism
vLLM recovery	`vllm_react_recovery_arm_v2`	DeepSeek brain, local Qwen vLLM backend	C={2, 4}, 3 repeats	Recovered sessions, avoided replay	Feasibility
HTTP+SQLite E2E	`business_workflow_e2e_smoke_v1` / `business_workflow_e2e_perf_sanity_v1`	External HTTP services and SQLite	9 + 18 rows	Idempotency, DB consistency	Correctness
CloudLab	`cloudlab_business_workflow_distributed_deterministic_v1`	Six nodes, Redis vs memory	Forced cross-gateway probes	State miss, recovery	Distributed correctness
Operator overhead	`operator_overhead_v1`	Same gateway/proxy path	C={20, 50, 100}, 5 repeats	p95/p99, throughput, CPU/RSS	Deployability
Sensitivity	`config_sensitivity_v1`	Controlled workflow grid	126 rows	Recovery, cascade	Robustness

result bundle, role, primary metrics, support level, caveat, allowed use, and prohibited interpretation. Repeated experiments report means, deltas, and bootstrap confidence intervals where appropriate; deterministic probes and single-repeat smoke tests are interpreted descriptively as correctness, diagnostic, or scope checks.

Table 5 summarizes the mapping from research questions to studies. Numeric results are deferred to the Results section so that methodology defines how each study should be interpreted before reporting the findings.

The submitted package supports lightweight replay before full reruns. The validator checks file presence, run metadata, statistical summaries, deterministic CloudLab invariants, HTTP and SQLite sanity invariants, and absence of leaked runtime files. Full experiment reruns remain available through scripts, while lightweight checks verify that the submitted package is internally consistent.

7.4. Artifact Availability and Replay Validation

The artifact is organized as a versioned package rather than as a single monolithic run directory. The submitted GitHub branch contains the gateway implementation, experiment runners, generated figures, and result summaries. The canonical statistical bundle for the JSS manuscript is the versioned package `statistical_summary_v4`, which contains the unified statistical summary, effect-size summary, and finding table used to constrain the paper's numerical statements. A lightweight validator checks required directories, run metadata, summary row counts, duplicate-side-effect invariants, deterministic CloudLab recovery invariants, and the absence of transient runtime outputs.

Reruns are grouped by resource need. Mock and HTTP+SQLite sanity runs are lightweight. vLLM reruns require a local OpenAI-compatible vLLM endpoint and model weights. Cloud-provider smoke tests require provider credentials and are used only as scope checks. CloudLab reruns require a multi-node allocation. This separation is important for JSS reproducibility: the paper distinguishes checks that replay quickly from studies that are reproducible but infrastructure-dependent.

8. Results

The results are organized by governance finding rather than by run directory. Each subsection states the supported result, the primary study, and the intended scope. Controlled business-workflow experiments provide the main waste-reduction result; recovery probes and HTTP+SQLite checks support side-effect correctness; CloudLab checks distributed checkpoint availability; cloud-provider and trace-shaped runs are used as scope or realism checks rather than headline performance rankings. Table 9 summarizes the main findings and their interpretation limits.

8.1. RQ1: Service-Workflow Waste Reduction

The main service-governance result comes from the controlled business-workflow experiment. Under the high-load failure setting (C=100, F=0.2), adaptive capacity admission reduces cascade failures relative to no-capacity-step0 by 31.8 sessions, with a bootstrap confidence interval of [−43.6, −20.0]. It also reduces wasted service time by approximately 544,149 ms, with confidence interval [−673,121, −414,984] ms. Figure 3 shows this admission trade-off.

This result supports RQ1 with an important scope condition. In the same setting, raw workflow success is not the primary optimization target and may differ across policies. We therefore read the result as waste-governance support: when backend capacity and failures make admitted-but-doomed sessions likely, session-aware capacity admission reduces cascade waste and wasted service time.

The request-level baseline shows the same failure mode directly: at C=100, F=0.2, a per-request queue limiter completes 26.4 sessions on average, slightly above adaptive's 25.0, but raises cascade failures to 74.6 and wasted service time to 10,573 ms, versus 5.2 and 1,218 ms for adaptive governance, because it cannot distinguish a new request from a late continuation that would discard a completed workflow prefix.

Table 6 makes the reporting scope explicit. The 544,149 ms number is the aggregate adaptive-versus-no-capacity delta from the core service-workflow artifact. The 10,573 ms versus 1,218 ms comparison comes from the request-level baseline study, which reuses the same workflow definition but reports its own comparator row. The two rows support the same direction of effect without being numerically additive.

8.2. RQ2: Adaptive Admission Trade-Off

RQ2 asks whether capacity admission should be fixed or pressure-aware. The constrained business-workflow result in RQ1 shows one side of the trade-off: removing capacity step-0 rejection creates more cascade and wasted service time under high-load failure conditions. Strict admission captures the opposite risk: rejecting early can deny work that might have completed when the backend has headroom. Adaptive admission keeps hard rejection but changes only the capacity gate according to pressure signals.

The cloud-backend smoke test shows the other side of the trade-off. In the single-repeat cloud-provider setting, no-capacity-step0 exceeds adaptive by 7 successful sessions. This is a descriptive scope result rather than a global ranking: provider

Table 5: Evaluation evidence map for the JSS manuscript.

RQ	Primary evidence role	Supporting evidence	Main boundary
RQ1	Core service-workflow waste reduction	Controlled business workflows; adaptive ablation; vLLM support	Do not claim raw-success dominance.
RQ2	Adaptive admission trade-off	Strict/adaptive/no-capacity variants across constrained and provider-backed settings	Provider elasticity can invert rankings.
RQ3	Recovery and side-effect safety	Protocol edge cases; vLLM recovery arm; HTTP+SQLite checks	Not transparent server-side replay.
RQ4	Deployability and mechanism interpretation	Operator overhead; component ablation; configuration sensitivity; artifact replay validation	Not every component improves raw success.
RQ5	External validity and distributed correctness	HTTP+SQLite sanity; MCP-Bench smoke; BurstGPT-shaped replay; CloudLab deterministic recovery	No production Redis HA claim.

Table 6: Why the two wasted-service-time comparisons should not be merged numerically. Both rows support the same waste-reduction direction, but they come from different result bundles and comparator definitions.

Comparison	Source artifact	Reported quantity	Adaptive	Comparator	Delta
Adaptive vs no-capacity-step0	`business_workflow_service_v3`	Mean wasted service time / run at C=100, F=0.2	1,218 ms	545,367 ms	−544,149 ms
Adaptive vs request-level admission	`request_level_baseline_v1`	Mean wasted service time / run in the request-level baseline study at C=100, F=0.2	1,218 ms	10,573 ms	−9,355 ms

elasticity can absorb load that the local gateway would otherwise reject. Figure 4 summarizes this backend-dependent interpretation.

8.3. RQ3: ReAct Recovery and Side-Effect Safety

Client-cooperative recovery improves recovered progress without turning the gateway into an automatic replay engine. In the core business-workflow setting at C=100, F=0.2, adaptive-react-recovery increases recovered ReAct sessions by 2.4 relative to adaptive admission alone, with confidence interval $[0.8, 4.0]$. It also increases avoided replay steps by 10.0, with confidence interval $[1.6, 19.6]$.

The targeted vLLM recovery arm provides model-backed feasibility evidence. At C=2, the recovery variant increases recovered sessions by 9.33 relative to adaptive, with confidence interval $[8.0, 10.0]$. This arm uses a hybrid setup with a DeepSeek agent brain and a local Qwen vLLM backend, so it should not be merged with natural full-vLLM recovery claims. Figure 5 summarizes the recovery results.

An idempotency-only retry baseline clarifies the boundary. Stable idempotency keys keep duplicate-side-effect count at zero, but without resume it records zero recovered sessions and zero avoided replay steps; completed prefixes are replayed instead.

Protocol-level checks support the same scope. The recovery protocol passes 11/11 declared edge cases, including no automatic future-tool execution and duplicate-side-effect checks. The E2E correctness smoke contains 9 summary rows with empty validation errors. The E2E performance-sanity set contains 18 runs over $C \in \{10, 20\}$, zero duplicate side effects, and consistent database state in 18/18 rows. These checks support side-effect correctness and recovery usability, not real-service performance ranking.

8.4. RQ4: Deployability, Overhead, and Sensitivity

The operator-overhead study indicates bounded overhead in the tested gateway path. At C=100, adaptive versus baseline has a small negative p95 latency delta in this run, approximately −6.37 ms, while the throughput delta confidence interval crosses zero. The same table also shows the adverse case that should not be hidden: adaptive-only raises p99 from 161.6 ms to 201.3 ms in this run, whereas adaptive+recovery does not show the same rise and remains at 161.7 ms with comparable throughput. We therefore interpret the artifact as bounded overhead within the tested gateway/proxy path, not as a general latency-improvement result.

Table 8 reports the C=100 resource summary. The baseline is the same gateway/proxy path rather than direct-to-backend execution. The table therefore measures incremental mechanism overhead within the gateway path, not the full cost of inserting a gateway into an otherwise direct deployment. This failure-free artifact does not trigger recovery and should not be used as recovery-effectiveness evidence.

The configuration-sensitivity study shows that governance parameters affect recovery strength. Under C=100, F=0.2, a strict-versus-default recovery-sensitivity comparison changes recovered sessions by −1.67. This supports qualitative robustness and sensitivity analysis, not selection of a globally optimal configuration. Component mapping records 9 architecture components mapped to 18 result records, which is used to organize the design rationale rather than to argue that every component improves raw success. Figure 6 summarizes overhead and sensitivity results.

8.5. RQ5: External Validity and Distributed Correctness

The real-service and distributed studies provide the main external-validity support. The HTTP+SQLite checks show that

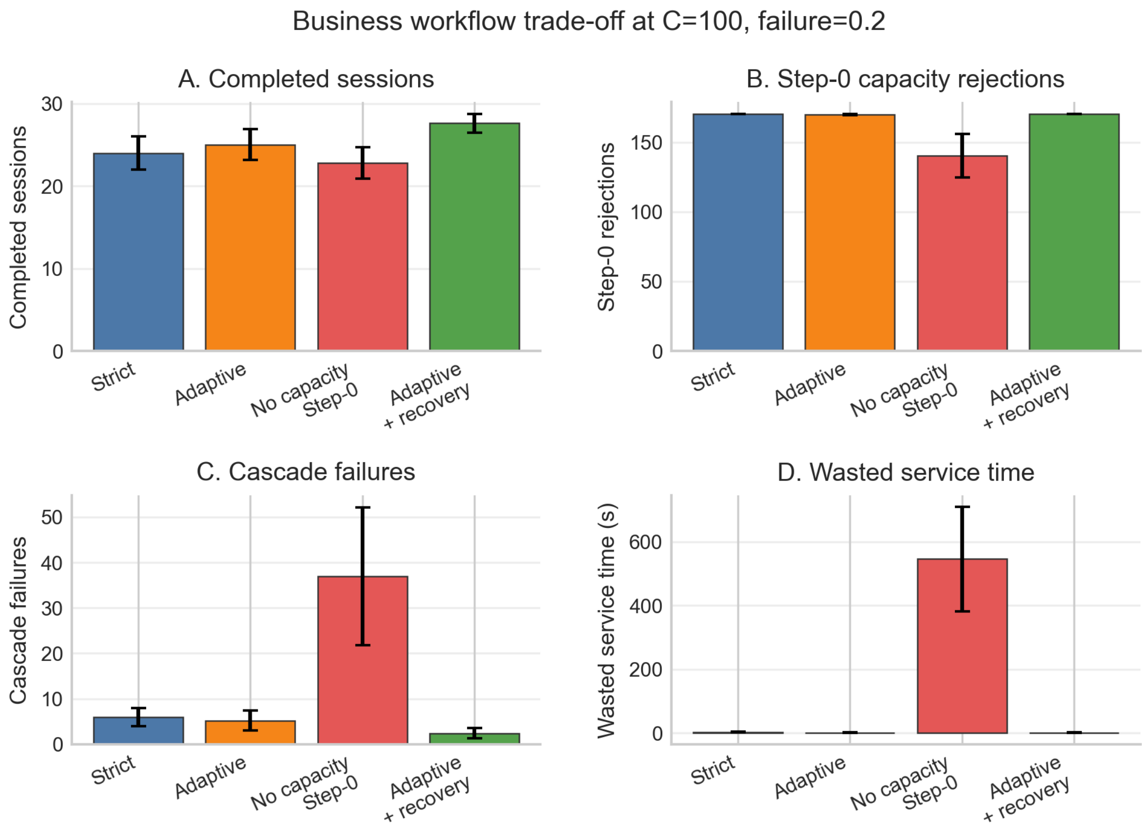

Figure 3: Core service-workflow trade-off at C=100, F=0.2. The panels show completed sessions, step-0 capacity rejections, cascade failures, and wasted service time. The figure reports an admission trade-off rather than a raw-success ranking: adaptive admission reduces cascade and wasted service time under constrained capacity, while early rejection can reduce raw completions in some settings.

Table 7: Idempotency-only retry versus checkpoint recovery at C=20, F=0.1.

Variant	Duplicate side effects	Resume recovered	Avoided replay steps	Replayed completed steps	Interpretation
Idempotency-only retry	0.0	0.0	0.0	8.0	Protects durable writes, but does not restore progress.
Adaptive + ReAct recovery	0.0	4.0	6.0	0.0	Preserves the safe prefix and avoids replay after resume.

MCP tools call external HTTP services, side effects are persisted, idempotency prevents duplicate writes, and recovery can resume without replaying completed operations. These runs are treated as correctness and sanity checks rather than the main performance setting.

The clean CloudLab distributed result is the deterministic correctness probe. CloudLab provides scientific infrastructure for controlled cloud-architecture experiments [27]. In our six-node deployment, Redis-backed shared state recovers 15/15 forced cross-gateway ReAct resumes with zero state misses, while the memory-local diagnostic control misses 15/15 forced cross-gateway resumes. Replayed completed side effects are zero. Figure 7 shows the deployment topology, and Figure 8 shows the distributed correctness result.

An earlier CloudLab distributed probe provides supporting context: Redis recovered 9/9 forced cross-gateway resumes, while memory-local state missed 9/9. The deterministic study supersedes this earlier mixed random-workload run for the main distributed-correctness result because the earlier random workload rows include expected ACTIVE_CHECKPOINT conflicts.

8.6. *Workflow Shape, Arrival Realism, and Cross-Layer Smoke*

Supporting studies address representativeness and execution stability. A metadata-derived MCP-Bench smoke selects 30 tasks and executes without errors, supporting workflow-shape diversity rather than full MCP-Bench deployment or model-accuracy conclusions. A BurstGPT-shaped replay covers normal, burst, and peak-burst windows with scale factors 1.0, 1.5, and 2.0, supporting arrival-pattern realism while keeping session structure controlled. The cross-layer smoke index covers mock, cloud LLM, and vLLM layers with empty validation errors. These studies support generality and stability, not headline performance rankings.

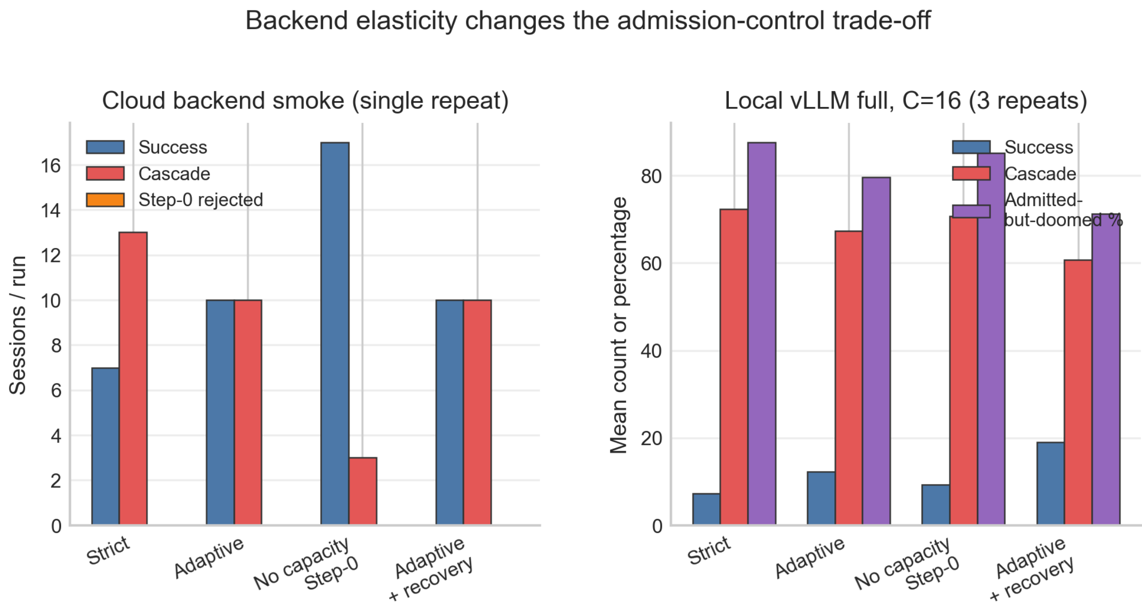

Figure 4: Backend-dependent admission scope. The panels are boundary illustrations, not a paired comparison: the left panel is a single-repeat cloud backend smoke test, while the right panel is a repeated local vLLM full run at C=16 with three repeats. They illustrate how provider elasticity can make no-capacity admission favorable in the cloud smoke, while constrained local vLLM exposes cascade and admitted-but-doomed pressure.

Table 8: Operator overhead at C=100 in the failure-free overhead artifact.

Variant	p95 ms	p99 ms	Throughput	CPU Δs	Avg RSS MB	Peak RSS MB
Baseline gateway	86.3	161.6	552.1	3.92	34.4	41.2
Adaptive	79.9	201.3	551.3	3.98	37.2	46.3
Adaptive + recovery	88.0	161.7	552.8	4.41	37.3	46.0

9. Discussion

9.1. Why Session-Aware Governance Helps

The results support the paper's central claim: request-level governance is insufficient for dependent agent workflows. When capacity is constrained, admitting a session that later fails wastes completed tool calls and service time. Adaptive capacity admission reduces this waste by moving some unavoidable rejection earlier while preserving hard rejection for invalid workflows. Recovery contributes a second benefit: it restores a safe point without making the gateway an autonomous agent or replay engine.

The request-level baseline is useful here because it can reject or pass individual calls without looking clearly worse in raw success, yet still accumulates much larger cascade and wasted service time because it has no model of completed prefix value. That gap is the practical difference between request-level and session-level governance. Idempotency alone protects durable writes; it does not preserve workflow progress.

9.2. When Admission Control Can Hurt

The provider-backed results show that admission control is not universally beneficial. Elastic cloud providers can hide queueing pressure, absorb load bursts, and make no-capacity-step0 look better in raw success. This defines the deployment scope: PlanGate is most valuable when backend capacity is constrained, shared, costly, or stateful. When a provider absorbs overload externally, the gateway should use provider-aware signals rather than enforce a fixed local capacity rule.

9.3. Distributed State as a Correctness Requirement

The CloudLab results show that shared state is not merely an optimization. Cross-gateway resume requires checkpoint availability beyond a single process. Sticky routing can preserve locality during ordinary traffic, but once a resumed session reaches a different gateway it cannot replace shared checkpoint lookup. The memory-local diagnostic control fails exactly where expected: it cannot find checkpoints created on another gateway. Redis-backed state fixes this lookup problem in the tested deployment. The result should still be read narrowly. It validates shared-state necessity for checkpoint and reservation lookup; it does not claim production Redis high availability, crash recovery, or replicated control-plane fault tolerance.

9.4. Implications for Service Workflow Engineering

The broader implication is that LLM-agent infrastructure should expose workflow semantics to the gateway layer. Session identifiers, step indexes, idempotency keys, recovery modes, and declared-plan metadata are not incidental annotations. PlanGate turns this metadata into an operational control contract

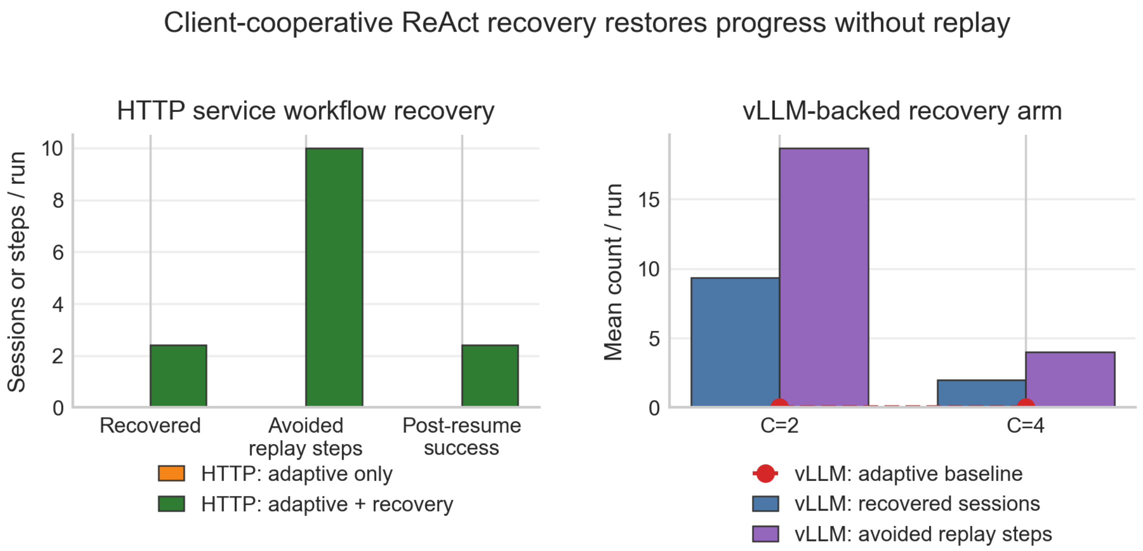

Figure 5: Client-cooperative ReAct recovery. The left panel reports HTTP service-workflow recovery; the right panel reports the targeted vLLM recovery arm. Recovery restores checkpointed progress and avoids replayed steps, but it does not execute future tools on behalf of the client.

Table 9: Main JSS results and interpretation scope.

Role	**Supported result**	**Scope**
Core service workflow	Adaptive reduces cascade by 31.8 and wasted service time by about 544,149 ms at C=100, F=0.2.	Not a raw-success ranking.
Request-level baseline	Per-request queue admission keeps raw completion comparable, but raises cascade to 74.6 and wasted service time to 10,573 ms at C=100, F=0.2.	Baseline for request-vs-session governance, not universal gateway ranking.
Client-cooperative recovery	Recovery adds 2.4 recovered sessions and 10.0 avoided replay steps in the core workflow.	Does not auto-execute future tools.
Real-backend recovery	vLLM recovery arm adds 9.33 recovered sessions at C=2.	Targeted hybrid arm, not natural full-vLLM recovery.
Idempotency-only baseline	Stable idempotency keys keep duplicate side effects at zero, but recovered sessions and avoided replay stay at zero while completed prefixes are replayed.	Separates duplicate-write safety from progress recovery.
E2E side-effect correctness	HTTP+SQLite sanity records zero duplicate side effects and consistent DB state in 18/18 rows.	Small sanity, not main performance benchmark.
Distributed correctness	CloudLab Redis recovers 15/15 forced cross-gateway resumes; memory misses 15/15.	No production Redis HA claim.
Provider scope	Cloud backend smoke shows no-capacity can exceed adaptive by 7 successes.	Single-repeat descriptive result.

through which the gateway can reason about cascade waste, continuation value, side-effect safety, and distributed recovery.

10. Threats to Validity

10.1. Internal Validity

The controlled experiments use repeated runs and fixed configurations, but they still depend on implementation choices in the runners and failure injectors. Bugs in clients, validators, or aggregation scripts could affect the results. To reduce this risk, the package includes run metadata, bootstrap/effect summaries, scoped finding tables, and a lightweight replay validator. Unit tests cover recovery semantics and session-cap behavior, but they do not eliminate all implementation risk.

10.2. Construct Validity

Metrics such as cascade failure, wasted service time, avoided replay steps, duplicate side effects, and state miss are proxies for service-workflow governance quality. They capture the paper's main failure modes, but they do not capture all possible user-facing outcomes. For example, raw workflow success can be lower under admission control even when cascade waste is lower. The paper therefore avoids interpreting all metrics as a single scalar utility.

10.3. Client Cooperation and Metadata Validity

PlanGate relies on client-provided workflow metadata, including session identifiers, step indexes, idempotency keys, and recovery modes. The evaluation checks malformed metadata and recovery edge cases, but it does not cover a complete adversarial-client model. The prototype mitigates accidental or opportunistic misuse through DAG validation, bounded amendment, session-cap limits, commitment and recovery token checks, and idempotency keys, but these controls do not provide Byzantine security against malicious clients. A client that omits metadata, reuses identifiers incorrectly, or intentionally lies about workflow state can weaken the governance contract. This threat is

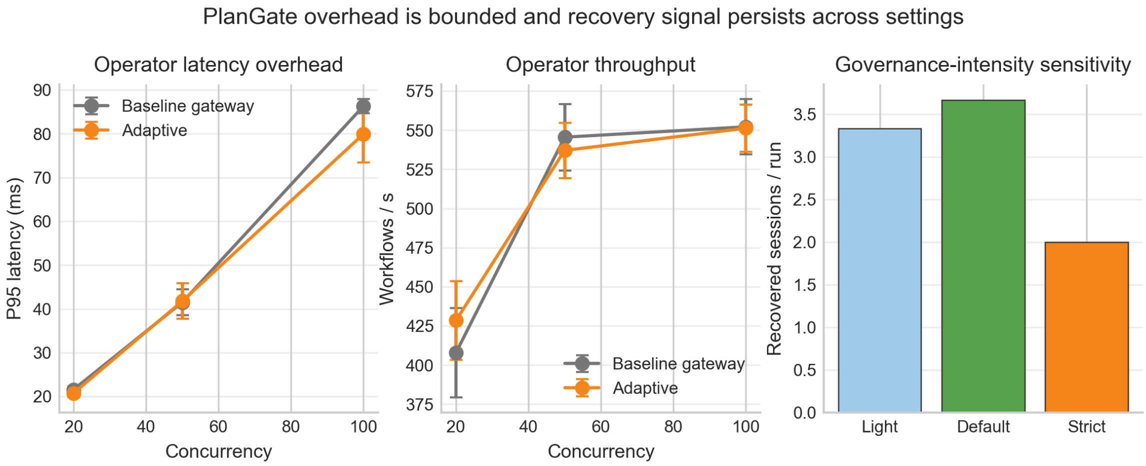

Figure 6: Deployability results. In the tested gateway/proxy path, p95 and throughput remain comparable, but adaptive-only can still raise p99 in a specific run. Configuration sensitivity shows that recovery strength depends on governance settings.

inherent in making workflow metadata part of the operational interface.

10.4. External Validity

The evaluation covers controlled business workflows, mock backends, vLLM, cloud-provider smoke tests, HTTP+SQLite services, MCP-Bench-derived workflow shapes, BurstGPT-shaped arrival replay, and CloudLab distributed deployment. This is broader than a single benchmark, but it is not a production deployment study. The CloudLab experiments are correctness probes rather than long-running production workloads. The BurstGPT-shaped replay uses real LLM-serving arrival structure, but session structure remains controlled. Cloud-provider behavior may vary across time, region, quota, and provider policy.

10.5. Reproducibility Threats

Some experiments depend on external cloud providers and local vLLM deployment, which can be sensitive to model versions, rate limits, and hardware. AI-enabled software systems can accumulate hidden dependencies and operational technical debt, so uncontrolled provider changes are a real reproducibility threat [29]. The artifact addresses this by separating controlled studies from provider-scope smoke tests and by recording run metadata. The lightweight replay validator checks internal consistency of the submitted package, but full reruns may still require provider credentials, CloudLab resources, or local model-serving setup.

10.6. Cost-Model Validity

Wasted service time and token use are proxy costs. They are useful for comparing policies under controlled settings, but they do not directly equal provider billing, user-perceived utility, or business value. The paper therefore treats cost-related metrics as operational waste indicators rather than a complete economic model.

11. Related Work

11.1. LLM Agent Tool Use and MCP Workflows

Tool-using LLM research studies how models select tools, call APIs, and solve multi-step tasks. ReAct interleaves reasoning traces and actions [37]; Toolformer studies how language models can learn when and how to call tools [28]; ToolBench and Gorilla-style work evaluate large API/tool ecosystems [25, 24]. These systems and benchmarks are essential for measuring whether an agent chooses the right tool and supplies the right arguments. PLANGATE addresses a different layer: it governs the service workflow after tool calls are issued. Its focus is admission, continuation, recovery, idempotency, and distributed state rather than model reasoning accuracy.

Recent agent benchmarks also show why gateway-level governance is needed. WebArena and OSWorld evaluate agents in realistic browser or computer environments [40, 36]. τ-bench models user-agent-tool interaction in business domains, and MCP-Bench connects agents to live MCP servers and hundreds of tools [38, 33]. These benchmarks emphasize task completion and tool-use competence. In this paper, they motivate workflow-shape smoke tests, but they are not the main performance substrate. The core claim is about infrastructure governance under constrained or shared backends, not about agent task accuracy.

MCP standardizes the connection between model applications and external tools [1]. This standardization makes the gateway position especially important: the gateway can observe tool calls, session metadata, and backend outcomes without modifying the model. PLANGATE uses this position to enforce session-aware governance, but it does not require a new agent framework or model-specific tool-use training.

11.2. Agent Frameworks, Workflow Engines, and Gateways

Agent workflow frameworks such as LangGraph, AutoGen, Semantic Kernel, and CrewAI help developers construct multi-agent applications, coordinate tool use, and represent agent state

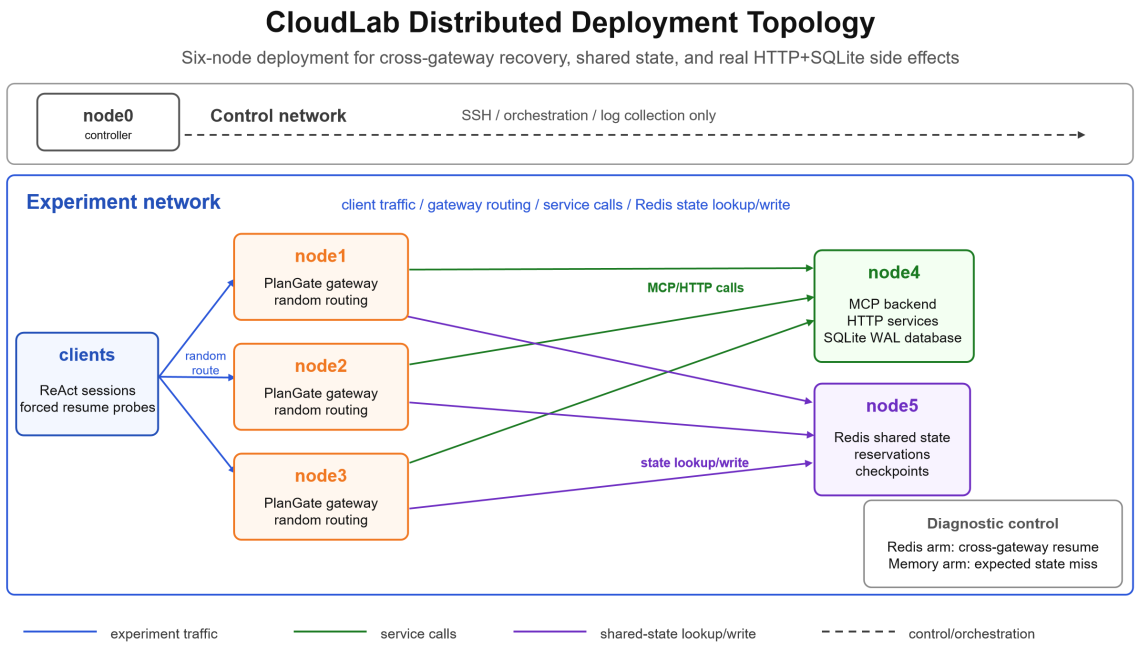

Figure 7: CloudLab distributed deployment topology used for the deterministic correctness probe. Multiple gateways route to external business services, while Redis and memory-local state are compared as shared-state and diagnostic arms.

[6, 4]. They help define or execute agent logic; PlanGate governs the infrastructure path after tool calls reach the gateway. Service waste, admission pressure, checkpoint lookup, and side-effect replay can arise even when the agent framework chooses tools correctly.

Workflow engines such as Temporal, Apache Airflow, Cadence, and Azure Durable Functions provide durable orchestration, scheduling, retries, and workflow state management [8, 2]. PlanGate does not replace them. It is a lighter gateway mechanism for agent-generated MCP workflows where the agent or application remains responsible for future tool choice.

API gateways and service meshes such as Envoy, Kong Gateway, Istio, and Linkerd provide routing, load balancing, retries, observability, and policy enforcement for service traffic [3, 5]. Their policies are usually request- or route-oriented. PlanGate instead makes the dependent session explicit: a late continuation step has different waste and recovery implications from a new session at step 0.

LLM gateway and router systems such as LiteLLM and Portkey focus on provider normalization, model routing, cost tracking, and OpenAI-compatible proxy behavior [7]. PlanGate addresses a different layer: multi-tool session lifecycle, side-effect safety, and client-cooperative ReAct recovery across backend services.

11.3. Service Workflows and Runtime Governance

Traditional service workflow systems reason about orchestration, compensation, state, and reliability across dependent service calls. Service-oriented computing frames services as composable software units and identifies composition, coordination, and management as core research concerns [22, 23]. Saga-style transactions define long-lived activities and compensating actions [14]. Workflow research and workflow-pattern studies characterize recurring control-flow, synchronization, and exception-handling structures in service processes [30]. These systems often assume an explicit workflow specification or an orchestration engine. PlanGate is lighter weight: it does not introduce a new workflow language and does not require all agents to submit a complete workflow graph. Instead, it uses metadata available at the gateway to govern admission, continuation, and recovery for agent-generated service calls.

API gateways, load balancers, rate limiters, and overload controllers protect backend services by shaping request traffic. Classical server overload work, staged event-driven architectures, autonomic management, and queue-management mechanisms show that runtime policy must account for queueing, backpressure, and adaptation [34, 17, 20]. Auto-scaling surveys similarly show that resource policies depend on workload, SLA, and elasticity assumptions [19]. These systems usually treat requests, queues, or resource pools as the control unit. PlanGate differs by making the session the governance unit: late-step rejection has a different cost from new-session rejection, and recovery must account for completed side-effecting steps. This is the gap that motivates session-aware governance.

Microservice research highlights independently deployable services, distributed service interactions, and operational complexity [12]. Benchmarks such as DeathStarBench and TeaStore provide production-like service graphs for studying latency, resource management, and service interactions [13, 31]. PlanGate is complementary. It does not replace microservice resource management; instead, it governs LLM-agent workflows that call tools and services, including model-backed and side-

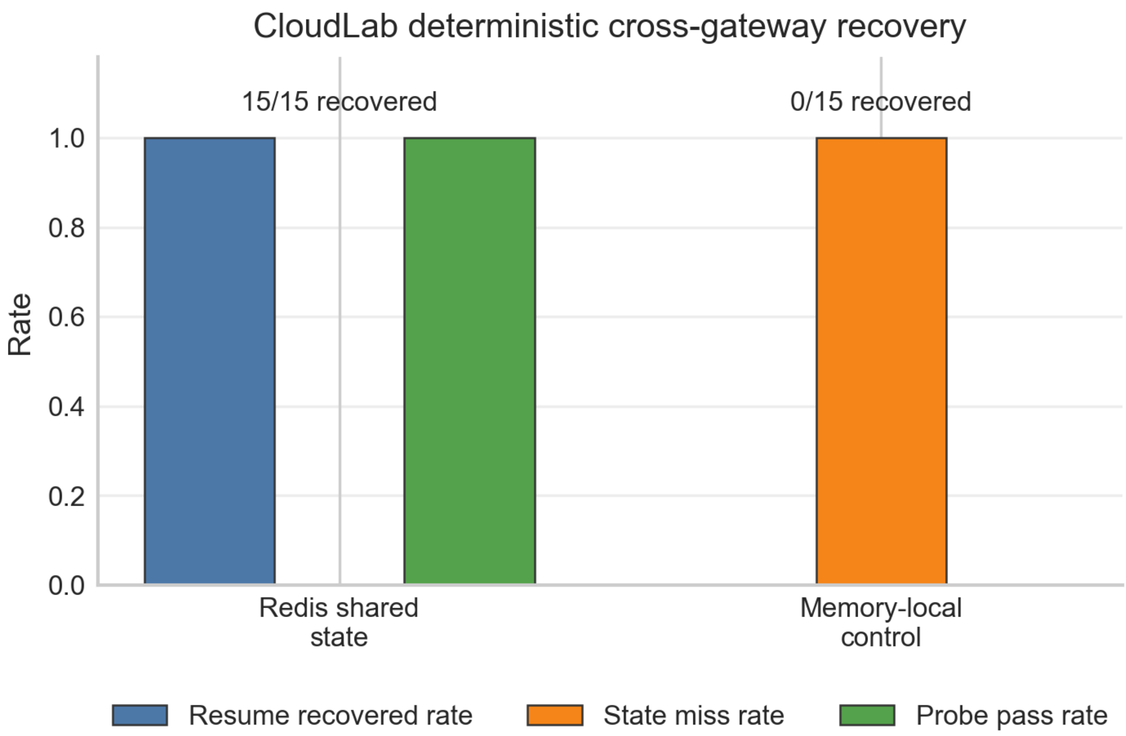

Figure 8: Distributed correctness on CloudLab. Redis-backed shared state supports forced cross-gateway recovery; memory-local state misses cross-gateway checkpoints. This checks shared-state necessity, not production Redis HA.

effecting services.

11.4. LLM Serving and Backend Overload

LLM serving systems study batching, scheduling, caching, and inference throughput. Orca introduces iteration-level scheduling for transformer serving, vLLM uses PagedAttention to improve memory management, and FastServe explores distributed inference scheduling for LLMs [39, 18, 35]. These systems are complementary to PlanGate. A serving system decides how to execute model requests efficiently; PlanGate decides whether and how a multi-step workflow should be admitted, continued, or recovered across tools and services that may include model-backed calls. The vLLM experiments in this paper use local model serving as a constrained backend, but the mechanism is gateway-level and backend-agnostic.

Trace-based LLM-serving studies such as BurstGPT show that model-serving traffic can be bursty and session-shaped [32]. Tail-latency studies also show that queueing and fan-out effects can dominate perceived service performance at scale [11]. PlanGate uses trace-shaped replay as supporting arrival-realism evidence, but it does not claim to possess a production agent trace. Cloud LLM providers add another layer: hidden queues, elastic headroom, rate limits, and provider-specific failure behavior. The provider-boundary results show that policies tuned for local constrained backends may not maximize raw success under elastic providers. This observation aligns with service-management work that treats runtime policy as environment-dependent rather than fixed.

11.5. Recovery, Idempotency, and Distributed State

Distributed systems commonly use checkpointing, idempotency, leases, durable logs, and shared state to handle failures. Chandy-Lamport snapshots define a classic basis for consistent distributed-state observation [9]. Leases provide time-bounded coordination [15]; ZooKeeper provides coordination primitives for distributed systems [16]; Dynamo and Raft illustrate different approaches to availability and consensus in distributed storage and replication [10, 21]. PlanGate does not attempt to solve full distributed consensus or replicated storage. It uses shared state narrowly for reservation and checkpoint lookup across gateways.

Service systems also rely on idempotency keys and stable operation identifiers to make retries safe. HTTP semantics distinguish safe and idempotent operations, but practical workflows still require application-level identifiers for durable side effects [26]. PlanGate applies these ideas to LLM-agent service workflows, where the failure boundary is complicated by online tool choice and side-effecting operations.

The recovery protocol is deliberately client-cooperative because future tool choice belongs to the agent client. Redis-backed state is used to make checkpoints available across gateway processes, while the paper avoids claims about production-grade high availability. This positions PlanGate between stateless API gateways and full workflow engines: it adds enough session state to govern agent workflows without taking over the agent's planning role.

Table 10: Positioning of PlanGate relative to adjacent research lines.

Line of work	Primary focus	Gap addressed by PlanGate
Agent/tool benchmarks	Tool choice and task success.	Gateway-level workflow governance after tool calls are issued.
LLM serving	Efficient model-request execution.	Session admission, continuation, and recovery across tools.
Workflow engines	Explicit orchestration and compensation.	Lightweight governance for agent-generated MCP workflows.
Distributed state	Coordination and storage correctness.	Cross-gateway checkpoint availability for agent sessions.

11.6. Positioning Summary

Table 10 summarizes the distinction between PlanGate and adjacent research lines. The common theme is that prior work either evaluates the agent, optimizes the backend, orchestrates explicit workflows, or provides distributed coordination. PlanGate instead targets gateway-level governance for multi-step service workflows after tool calls are issued but before their execution and recovery semantics become unsafe.

12. Conclusion

This paper presented PlanGate, a session-aware governance gateway for multi-step LLM service workflows. The central observation is that request-level governance does not match dependent agent sessions: rejecting a late tool call can waste earlier work and risk unsafe replay. PlanGate addresses this mismatch with session-aware admission, client-cooperative recovery, idempotency, and optional Redis-backed shared state.

The evaluation shows that adaptive governance reduces cascade and wasted service time under constrained business-workflow load, ReAct recovery restores checkpointed progress without server-side replay, and HTTP+SQLite and CloudLab studies validate side-effect and distributed-state correctness. The results also define a deployment scope: elastic cloud providers can invert admission-control trade-offs. PlanGate should therefore be read as a workflow-governance framework for constrained, shared, or stateful service backends, not as a universally dominant admission policy.

References

[1] Anthropic. Model Context Protocol. `https://modelcontextprotocol.io/`, 2024.

[2] Apache Airflow. Apache Airflow documentation. `https://airflow.apache.org/docs/`, 2026.

[3] Envoy Proxy. Envoy documentation. `https://www.envoyproxy.io/docs/`, 2026.

[4] Microsoft AutoGen. AutoGen documentation. `https://microsoft.github.io/autogen/`, 2026.

[5] Istio. Istio documentation. `https://istio.io/latest/docs/`, 2026.

[6] LangChain. LangGraph documentation. `https://langchain-ai.github.io/langgraph/`, 2026.

[7] LiteLLM. LiteLLM documentation. `https://docs.litellm.ai/`, 2026.

[8] Temporal Technologies. Temporal documentation. `https://docs.temporal.io/`, 2026.

[9] K. M. Chandy and L. Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.

[10] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels. Dynamo: Amazon's highly available key-value store. In *Proceedings of SOSP*, 2007.

[11] J. Dean and L. A. Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[12] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina. Microservices: Yesterday, today, and tomorrow. In M. Mazzara and B. Meyer (Eds.), *Present and Ulterior Software Engineering*, pp. 195–216. Springer, 2017.

[13] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, and C. Delimitrou. An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems. In *Proceedings of ASPLOS*, 2019.

[14] H. Garcia-Molina and K. Salem. Sagas. In *Proceedings of ACM SIGMOD*, 1987.

[15] C. Gray and D. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of SOSP*, 1989.

[16] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed. ZooKeeper: Wait-free coordination for Internet-scale systems. In *Proceedings of USENIX ATC*, 2010.

[17] J. O. Kephart and D. M. Chess. The vision of autonomic computing. *Computer*, 36(1):41–50, 2003.

[18] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of SOSP*, 2023.

[19] T. Lorido-Botran, J. Miguel-Alonso, and J. A. Lozano. A review of auto-scaling techniques for elastic applications in cloud environments. *Journal of Grid Computing*, 12:559–592, 2014.

[20] K. Nichols and V. Jacobson. Controlling queue delay. *Communications of the ACM*, 55(7):42–50, 2012.

[21] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of USENIX ATC*, 2014.

[22] M. P. Papazoglou. Service-oriented computing: Concepts, characteristics and directions. In *Proceedings of WISE*, 2003.

[23] M. P. Papazoglou, P. Traverso, S. Dustdar, and F. Leymann. Service-oriented computing: A research roadmap. *International Journal of Cooperative Information Systems*, 17(2):223–255, 2008.

[24] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. Go-

rilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

[25] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *Proceedings of ICLR*, 2024.

[26] RFC Editor. RFC 9110: HTTP Semantics. `https://www.rfc-editor.org/rfc/rfc9110`, 2022.

[27] R. Ricci, E. Eide, and the CloudLab Team. Introducing CloudLab: Scientific infrastructure for advancing cloud architectures and applications. *USENIX ;login:*, 39(6):36–38, 2014.

[28] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

[29] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, 2015.

[30] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros. Workflow patterns. *Distributed and Parallel Databases*, 14:5–51, 2003.

[31] J. von Kistowski, S. Eismann, N. Herbst, A. Bauer, J. Grohmann, and S. Kounev. TeaStore: A micro-service reference application for benchmarking, modeling and resource management research. In *Proceedings of the IEEE International Symposium on the Maintenance and Evolution of Service-Oriented and Cloud-Based Systems*, 2018.

[32] Y. Wang, Y. Chen, Z. Li, X. Kang, Y. Fang, Y. Zhou, Y. Zheng, Z. Tang, X. He, R. Guo, X. Wang, Q. Wang, A. C. Zhou, and X. Chu. BurstGPT: A real-world workload dataset to optimize LLM serving systems. *arXiv preprint arXiv:2401.17644*, 2024.

[33] Z. Wang, Q. Chang, H. Patel, S. Biju, C.-E. Wu, Q. Liu, A. Ding, A. Rezazadeh, A. Shah, Y. Bao, and E. Siow. MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers. *arXiv preprint arXiv:2508.20453*, 2025.

[34] M. Welsh, D. Culler, and E. Brewer. SEDA: An architecture for well-conditioned, scalable Internet services. In *Proceedings of SOSP*, 2001.

[35] B. Wu, Y. Zhong, Z. Zhang, G. Huang, X. Liu, and X. Jin. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.

[36] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V. Zhong, and T. Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, 2024.

[37] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of ICLR*, 2023.

[38] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

[39] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. Orca: A distributed serving system for Transformer-based generative models. In *Proceedings of OSDI*, 2022.

[40] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.